PinkSync System Technical Specification

1. System Architecture Overview

PinkSync is a bidirectional sign language authentication and communication system designed specifically for the Deaf community. The system incorporates multiple components including user management, authentication, translation, cultural integration, and visual-spatial information architecture.

1.1 Core Components

- **User Management System**: Handles user accounts, profiles, and permissions
- **Authentication Service**: Processes sign biometric verification and motion-based authentication
- **Translation Engine**: Bidirectional translation between sign and spoken languages
- **Cultural Integration Layer**: Processes regional variants and community-specific terminology
- **Spatial Grammar Processor**: Analyzes and interprets 3D signing space and non-manual markers
- **API Gateway**: Controls access to various system services
- **Webhook Service**: Enables third-party integrations and notifications
- **Data Storage**: Securely maintains user profiles, biometric templates, and system data

2. User Management

2.1 User Database Schema

```
User {
  id: UUID (primary key)
  username: String (unique)
  email: String (optional, unique if provided)
  phoneNumber: String (optional, unique if provided)
  fullName: String
```

```
  preferredSignLanguage: Enum [ASL, BSL, Auslan, etc.]
  regionalVariant: String (optional)
  dateCreated: Timestamp
  lastLogin: Timestamp
  status: Enum [ACTIVE, INACTIVE, SUSPENDED, PENDING_VERIFICATION]
  role: Enum [USER, INTERPRETER, ADMIN, DEVELOPER]
  authMethod: Enum [SIGN_BIOMETRIC, MOTION_PASSWORD, EMAIL_PASSWORD, SOCIAL, 2FA]
  preferredDevices: Array<DeviceID>
  accessPermissions: Array<Permission>
  communicationPreferences: Object
  visualSettingsPreference: Object
}

UserBiometrics {
  userId: UUID (foreign key to User.id)
  biometricTemplates: Array<EncryptedTemplate>
  motionPasswordHash: EncryptedString
  templateVersion: String
  lastUpdated: Timestamp
  failedAttempts: Integer
  lockoutStatus: Boolean
}

UserSession {
  sessionId: UUID (primary key)
  userId: UUID (foreign key to User.id)
  deviceId: String
  ipAddress: String
  userAgent: String
  startTime: Timestamp
  lastActive: Timestamp
  expiryTime: Timestamp
  authMethod: String
  status: Enum [ACTIVE, EXPIRED, TERMINATED]
}
```

2.2 User Registration Process

1. **Initial Signup**:
    - Basic information collection (name, email/phone, preferred sign language)
    - Terms of service acceptance
    - Generate temporary account with PENDING_VERIFICATION status
2. **Identity Verification**:

- Email/phone verification
- Optional: government ID verification for higher security levels

3. **Biometrics Enrollment**:
   - Capture sign language biometric samples (minimum 3 different sign phrases)
   - Generate and encrypt biometric template
   - Create motion-based password alternative
4. **Profile Completion**:
   - Communication preferences
   - Regional sign variant selection
   - Visual interface customization
   - Device registration

2.3 User Management API Endpoints

2.3.1 User Creation and Management

```
POST /api/v1/users
GET /api/v1/users/{userId}
PUT /api/v1/users/{userId}
PATCH /api/v1/users/{userId}
DELETE /api/v1/users/{userId} (soft delete)
```

2.3.2 User Search and Filtering

```
GET /api/v1/users?filter={filterParams}
GET /api/v1/users/search?q={searchTerm}
```

2.3.3 User Permissions and Roles

```
GET /api/v1/users/{userId}/permissions
POST /api/v1/users/{userId}/permissions
DELETE /api/v1/users/{userId}/permissions/{permissionId}
GET /api/v1/users/{userId}/roles
PUT /api/v1/users/{userId}/roles
```

2.3.4 User Status Management

```
PUT /api/v1/users/{userId}/status
GET /api/v1/users/{userId}/activity
```

3. Authentication System

3.1 Authentication Methods

1. **Sign Biometric Authentication**:
    - Real-time capture of signing pattern
    - Feature extraction and template matching
    - Confidence score calculation
    - Adaptive threshold based on security requirements
2. **Motion-Based Password Alternative**:
    - Predefined sequence of signs as password
    - Spatial-temporal pattern matching
    - Rhythm and movement characteristics analysis
3. **Visual Two-Factor Authentication (V2FA)**:
    - Traditional authentication (email/password, social login) as first factor
    - Sign biometric verification as second factor
    - Fallback options for accessibility
4. **Cross-Platform Authentication Persistence**:
    - Secure token generation and management
    - Device fingerprinting
    - Session synchronization across devices
    - Real-time session validation

3.2 Authentication API Endpoints

```
POST /api/v1/auth/sign-biometric
```

```
POST /api/v1/auth/motion-password
POST /api/v1/auth/login (traditional methods)
POST /api/v1/auth/logout
POST /api/v1/auth/refresh-token
GET /api/v1/auth/sessions
DELETE /api/v1/auth/sessions/{sessionId}
POST /api/v1/auth/verify-2fa
POST /api/v1/auth/reset-credentials
```

3.3 Biometric Template Management

```
GET /api/v1/users/{userId}/biometrics (metadata only)
POST /api/v1/users/{userId}/biometrics/enroll
PUT /api/v1/users/{userId}/biometrics/update
DELETE /api/v1/users/{userId}/biometrics/reset
```

4. API Groups and Endpoints

4.1 Core API Groups

1. **User Management API -** `/api/v1/users/*`
2. **Authentication API -** `/api/v1/auth/*`
3. **Translation API -** `/api/v1/translate/*`
4. **Cultural API -** `/api/v1/cultural/*`
5. **Spatial Grammar API -** `/api/v1/spatial/*`
6. **Visual Interface API -** `/api/v1/visual/*`
7. **Admin API -** `/api/v1/admin/*`
8. **Developer API -** `/api/v1/developer/*`
9. **Integration API -** `/api/v1/integrations/*`
10. **Analytics API -** `/api/v1/analytics/*`

4.2 Translation API Endpoints

```
POST /api/v1/translate/sign-to-text
POST /api/v1/translate/sign-to-voice
POST /api/v1/translate/text-to-sign
```

```
POST /api/v1/translate/voice-to-sign
GET /api/v1/translate/supported-languages
POST /api/v1/translate/batch
```

4.3 Cultural API Endpoints

```
GET /api/v1/cultural/variants/{signLanguage}
GET /api/v1/cultural/terminology/{community}
POST /api/v1/cultural/contribute
GET /api/v1/cultural/recommendations?user={userId}
```

4.4 Visual Interface API Endpoints

```
GET /api/v1/visual/layouts/{userId}
PUT /api/v1/visual/layouts/{userId}
POST /api/v1/visual/cognitive-load/optimize
GET /api/v1/visual/search?query={visualQuery}
```

4.5 Developer API Endpoints

```
POST /api/v1/developer/apps
GET /api/v1/developer/apps/{appId}
PUT /api/v1/developer/apps/{appId}
DELETE /api/v1/developer/apps/{appId}
POST /api/v1/developer/apps/{appId}/api-keys
DELETE /api/v1/developer/apps/{appId}/api-keys/{keyId}
GET /api/v1/developer/usage-stats
```

5. Webhook System

5.1 Webhook Registration

```
POST /api/v1/webhooks
GET /api/v1/webhooks
PUT /api/v1/webhooks/{webhookId}
DELETE /api/v1/webhooks/{webhookId}
POST /api/v1/webhooks/{webhookId}/test
```

5.2 Webhook Event Types

1. **User Events**:
    - `user.created`
    - `user.updated`
    - `user.login`
    - `user.logout`
    - `user.status_change`
2. **Authentication Events**:
    - `auth.success`
    - `auth.failure`
    - `auth.locked`
    - `auth.reset`
3. **Translation Events**:
    - `translation.completed`
    - `translation.failed`
    - `translation.feedback`
4. **System Events**:
    - `system.error`
    - `system.warning`
    - `system.update`

5.3 Webhook Payload Structure

```
{
  "event": "event.type",
  "timestamp": "ISO-8601 timestamp",
  "version": "1.0",
  "data": {
    "eventSpecificData": "value"
  },
  "metadata": {
    "userId": "UUID if applicable",
```

```
    "deviceInfo": "device information if applicable",
    "requestId": "request identifier for tracking"
  }
}
```

5.4 Webhook Security

- HMAC signature validation
- IP whitelisting
- Rate limiting
- Automatic suspension after consecutive failures
- Required HTTPS endpoints

6. CRUD Operations

6.1 Standard CRUD Patterns

All resources follow RESTful CRUD patterns:

- **Create**: POST /api/v1/{resource}
- **Read**: GET /api/v1/{resource}/{id}
- **Update**: PUT or PATCH /api/v1/{resource}/{id}
- **Delete**: DELETE /api/v1/{resource}/{id}

6.2 Batch Operations

For performance optimization with high-volume operations:

```
POST /api/v1/{resource}/batch
```

Payload structure:

```
{
  "operations": [
    {
      "method": "POST|GET|PUT|PATCH|DELETE",
      "resourceId": "optional-id",
      "data": {}
    }
```

```
  ],
  "options": {
    "continueOnError": true|false,
    "returnFailures": true|false
  }
}
```

6.3 Query Parameters

Standard query parameters across all GET operations:

- `fields` : Specify which fields to return
- `filter` : Filter criteria
- `sort` : Sort direction and field
- `page` and `limit` : Pagination controls
- `expand` : Expand related resources

7. Security Considerations and Risk Mitigation

7.1 Potential Security Risks

Risk Category	Specific Risks	Mitigation Strategies
Authentication Vulnerabilities	- Biometric template theft - Replay attacks - Motion pattern interception - Session hijacking	- Encrypted biometric templates - Liveness detection - Pattern variations tolerance - Secure session management
API Security	- Unauthorized access - Excessive permissions - Rate limiting bypass - API key leakage	- OAuth 2.0 with JWT - Granular permissions - Advanced rate limiting - API key rotation
Data Protection	- PII exposure - Data breach - Insecure data transmission - Inadequate encryption	- Data minimization - End-to-end encryption - TLS 1.3 - Proper key management
Webhook	- Webhook abuse - Credential exposure -	- Signature verification - Payload validation - Request

Security	Server-side request forgery	throttling
Infrastructure Security	- DDoS attacks - Server vulnerabilities - Dependency exploits - Container escapes	- CDN protection - Regular patching - Dependency scanning - Container security
Biometric-Specific Risks	- Presentation attacks - Deepfake videos - Adversarial examples - False match rates	- Multimodal verification - AI-based fake detection - Adversarial training - Threshold tuning
Cross-Platform Vulnerabilities	- Inconsistent security controls - Platform-specific exploits - Insecure data sync	- Security feature parity - Platform-specific security reviews - Encrypted synchronization

7.2 Authentication Security

7.2.1 Sign Biometric Security

- Template storage using non-reversible transformations
- Salted and peppered templates with key splitting
- Regular template rotation
- Anti-spoofing measures (liveness detection)
- Adaptive security level based on transaction risk
- Statistical anomaly detection for signing patterns

7.2.2 Motion Password Security

- Motion pattern hashing with argon2id
- Fuzzy matching with secure thresholds
- Progressive lockout after failed attempts
- Regular motion password rotation prompts
- Secondary verification for risky transactions

7.3 API Security Measures

- OAuth 2.0 with OpenID Connect for authentication
- JWT with appropriate expiration and refresh strategy
- API rate limiting based on user tier and endpoint sensitivity
- Input validation for all endpoints
- Response filtering to prevent data leakage
- API versioning for backward compatibility
- Comprehensive logging and monitoring

7.4 Data Protection

- Encryption at rest using AES-256
- Data categorization based on sensitivity
- PII anonymization and pseudonymization
- Data retention policies with automatic pruning
- Backup encryption and secure storage
- Regular security audits and penetration testing

7.5 Prevention of Common Attacks

7.5.1 Injection Protection

- Parameterized queries
- ORM with binding parameters
- Input sanitization
- Content Security Policy implementation
- Regular security testing

7.5.2 XSS Protection

- Content-Security-Policy headers
- Output encoding
- HTTPOnly and Secure cookie flags

- XSS filters

7.5.3 CSRF Protection

- Anti-CSRF tokens
- Same-site cookie policy
- Referrer checking
- Request origin verification

7.6 Software Development Security Practices

- Secure SDLC implementation
- Regular security training for developers
- Code review with security focus
- Automated security scanning
- Dependency vulnerability monitoring
- Container security scanning
- Regular penetration testing

7.7 Infrastructure Security

- Network segmentation
- Web Application Firewall (WAF)
- DDoS protection
- Intrusion Detection System (IDS)
- Regular vulnerability scanning
- Infrastructure as Code security validation
- Secure CI/CD pipeline

7.8 Monitoring and Incident Response

- Real-time security monitoring
- Anomaly detection

- User behavior analytics
- Comprehensive logging
- Incident response plan
- Regular tabletop exercises
- Post-incident reviews

8. Implementation Recommendations

8.1 Technology Stack

Component	Recommended Technologies	Rationale
Backend Services	- Node.js/Express - Python/FastAPI - Go for performance-critical services	Balances development speed with performance requirements
Authentication	- Auth0/Okta for standard auth - Custom modules for sign biometrics	Leverage proven solutions while adding custom capabilities
Database	- PostgreSQL for relational data - MongoDB for user profiles - Redis for caching - ClickHouse for analytics	Different data types require specialized storage
ML/Computer Vision	- TensorFlow - PyTorch - MediaPipe for skeletal tracking	Best-in-class frameworks for sign recognition
API Gateway	- Kong - AWS API Gateway - Tyk	Mature API management capabilities
Message Queue	- Kafka - RabbitMQ	Reliable asynchronous processing
Infrastructure	- Kubernetes - AWS/Azure/GCP services	Scalability and managed services

8.2 Development and Deployment Process

1. **Development Environment**:
    - Containerized development environments
    - Feature branch workflow
    - Pre-commit hooks for security and quality
2. **Testing Strategy**:
    - Unit testing (minimum 80% coverage)
    - Integration testing
    - Security testing (SAST, DAST)
    - Performance testing
    - Accessibility testing
3. **CI/CD Pipeline**:
    - Automated builds
    - Security scanning
    - Automated testing
    - Blue-green deployments
    - Canary releases
4. **Monitoring and Observability**:
    - Distributed tracing
    - Application performance monitoring
    - Real-time alerts
    - User experience monitoring
    - Security event monitoring

9. Implementation Phases

9.1 Phase 1: Core Infrastructure

- User management system

- Basic authentication (traditional + motion password)
- API gateway and basic rate limiting
- Core database architecture
- Development environments and CI/CD

9.2 Phase 2: Authentication Innovation

- Sign biometric capture and processing
- Motion password refinement
- Two-factor authentication implementation
- Session management across devices

9.3 Phase 3: Translation and Cultural Features

- Basic bidirectional translation
- Regional variant support
- Community terminology database
- Cultural integration layer

9.4 Phase 4: Developer Platform

- Developer portal
- API documentation
- Webhook system
- SDK development

9.5 Phase 5: Advanced Features

- Advanced visual-spatial interface
- Cognitive load optimization
- Machine learning improvements
- Additional platform integrations

10. Risk Assessment and Mitigation Plan

10.1 Technical Risks

Risk	Probability	Impact	Mitigation
Biometric accuracy below threshold	Medium	High	- Collect diverse training data - Implement backup authentication methods - Progressive improvement through feedback loops
Performance issues with translation	Medium	High	- Optimize critical code paths - Implement caching strategy - Consider edge computing for latency-sensitive operations
Scalability bottlenecks	Medium	Medium	- Load testing early - Design for horizontal scaling - Implement database sharding strategy
Third-party service dependencies	High	Medium	- Build abstraction layers - Implement circuit breakers - Have fallback mechanisms
Data migration complexities	Medium	Medium	- Comprehensive data modeling upfront - Version-controlled schema changes - Blue-green migrations

10.2 Security Risks

Risk	Probability	Impact	Mitigation
Biometric template leak	Low	Critical	- Zero-knowledge proof architecture - Encrypted template storage - Template separation from user data
API key compromise	Medium	High	- Short-lived keys - Automatic rotation - Usage pattern monitoring
Social engineering attacks	High	High	- Staff security training - Strict verification procedures - Limited support account access
Regulatory non-	Medium	High	- Regular compliance audits - Privacy impact assessments - Data protection

compliance			officer appointment
Zero-day vulnerabilities	Low	Critical	- Defense in depth - Regular penetration testing - Bug bounty program

10.3 Business Risks

Risk	Probability	Impact	Mitigation
User adoption barriers	Medium	Critical	- Early user testing - Comprehensive onboarding - Community ambassadors
Accessibility gaps	Medium	High	- Diverse user testing - Multiple interaction modes - Accessibility audit program
Integration challenges	High	Medium	- Well-documented APIs - Integration support team - Reference implementations
Resource constraints	High	Medium	- Phased implementation - Focus on core differentiators - Consider strategic partnerships
Competitive pressure	Medium	Medium	- Patent protection - Rapid innovation cycle - Community building

11. Compliance and Standards

11.1 Accessibility Standards

- WCAG 2.2 AAA compliance
- US Section 508 compliance
- EN 301 549 (European accessibility requirements)
- Custom Deaf-specific accessibility guidelines

11.2 Security Standards and Certifications

- SOC 2 Type II compliance
- NIST Cybersecurity Framework

- ISO 27001 certification
- GDPR compliance
- CCPA/CPRA compliance
- HIPAA compliance (if handling health information)

11.3 Biometric Standards

- ISO/IEC 24745 (Biometric information protection)
- ISO/IEC 19794 (Biometric data interchange formats)
- FIDO Alliance specifications
- Custom sign language biometric standards